

If We Build Them, They Will Run: Automated HPC Apps Deployment and Profiling with eBPF in Cloud

Vanessa Sochat and Daniel Milroy

Lawrence Livermore National Laboratory, Livermore CA 94550 USA

A Appendix

A.1 Applications

AMG2023 is provided via several builds across MPI variants, Ubuntu, and spack, and for CPU and GPU. We include AMG in our experiments, and ran problem 2 and used a problem size of 32x16x16 [?]. The figure of merit (FOM) is defined as:

$$FOM = \frac{nnz_AP}{SetupPhaseTime + 3 * SolvePhaseTime}.$$

In the above, *nnz_AP* is the total number of non-zeros, and two timings for the setup and the solving phase [?]. We ran AMG2023 tests in a weak scaling configuration.

Big Data Analytics Benchmarks (BDAS) [?] are a collection of data and machine learning (ML) scientific models implemented in R that use Message Passing Interface (MPI) for several well-known models (kmeans, svm, pca) that provide a holistic view of the capabilities of an environment. For our study we will run all three models with 250 rows, 50 columns for a problem size across scales. As was done in [?] we use the duration as the FOM.

cfdsCOPE (Computational Fluid Dynamics Scalability, Correctness, and Performance Estimator) is a set of mini-applications for computational fluid dynamics developed at TU Dresden that are intended for teaching [?]. The benchmarks run on one node, and we worked with the main author to develop a variant that would strong scale across threads (11, 22, 44, 88) for each of 5 iterations for our study. The FOM is the time to complete one simulation at each size.

chatterbug

We used the stencil3d benchmark, a part the LLNL Chatterbug communication proxy suite, to emulate the communication patterns of scientific applications performing stencil computations on a 3D structured grid. We used a grid size of 1024³ and 100 iterations and measured the time to complete a halo exchange [?]. This is a good library to evaluate general performance of scaled networking and MPI libraries.

e3sm kernels are standalone, computationally intensive core routines extracted from Earth system model components [?]. We will compile the Atmospheric kernels (the executable *atm*) using spack [?] to run across scales.

Laghos The LAGrangian High-Order Solver simulates the compression of gas using a moving frame [?]. We used the major kernels total rate (megadofs × time steps/second) as our FOM, and ran the app in a strong scaling configuration similar to the Vulcan example [?]

with a *cube_311_hex* mesh using partial assembly and a maximum of 400 steps.

LAMMPS is the Large-scale Atomic/Molecular Massively Parallel Simulator. It models the reaction of atoms and solves a matrix optimization problem. We are familiar with and thus used the *ReaxFF* package, following a suggested practice [?] to calculate the millions of atom steps per second for our FOM. A larger value is better, indicating a system can do more calculations per second. We chose a consistent problem size of 64x32x32 to run on GPU, and 64x64x32 for CPU for each of parameters *x*, *y*, and *z*, respectively. The GPU problem size was chosen to be smaller to fit on the GPUs on Google Cloud and *B*. We ran LAMMPS in a strong scaling configuration.

A.2 extended Berkeley Packet Filter (eBPF) Programs

tcp: We are interested to look at TCP-related system calls to get a glimpse of network activity, not just for our application, but other services running in Kubernetes that change with scale. Specifically, this program looks at data transfer sizes, call durations, and connection events. We again use tracepoints to attach to syscalls for sockets (connect, accept, close) and I/O calls (*sendto*, *recvfrom*, *read* and *write*), for each recording duration, return values, and bytes transferred. The data is sent to user-space via a ring buffer, where we again calculate RiverML statistics and counts for each. Since bytes transferred can fall within a large range, we group based on buckets of message sizes. From this program we can derive summary statistics for I/O operations broken down into buckets, including byte transfer and duration. If we see increased I/O call durations across scale, this could indicate the network stack taking longer to send and receive data, a possible result of the scaling. Higher latency or congestion that leads to longer syscall duration could mean increased round trip times or packet loss, contention within the TCP stack, or an application's slowness in receiving or processing data.

shared memory: The *shmem* "shared memory" program traces calls for several operations (e.g., *shmget* or *shm_open* or and *shm_unlink*), and calculates shared memory sizes to store in a BPF map keyed by the Process IDentifier (PID). The command name is also saved. The maps are periodically read in user space, and at program completion, summary of operation counts and map sizes is produced, group by PID and command. If we see increased shared memory usage as the application scales,

this could suggest a higher demand for inter-process communication or other bottlenecks.

A.3 BuildKit Configurations

Optimization Levels

We used BuildKit to generate containers across optimization levels *O0* (no optimization), *O1* (Basic optimizations), *O2* (standard optimization), *O3* (aggressive optimization), *Ofast* (aggressive, non-compliant math), *Os* (optimize for size), *Og* (optimize for debugging) for each of arm64 and amd64 platforms.

Micro-architectures

For amd64 we built native, x86-64-v2, x86-64-v3, x86-64-v4 (generic and portable based on instruction set levels), sandybridge, ivybridge, haswell, broadwell, skylake, skylake-avx512, icelake-server, sapphirerapids (Intel-specific), and btver2, bdver4, znver1, znver2, znver3 (AMD-specific). For arm64 we built native, neoverse-n1, neoverse-v1, neoverse-n2, neoverse-v2, ampere1, a64fx, cortex-a72, cortex-a53, cortex-a57, cortex-a76, cortex-x1, and generic micro-architectures.

Acknowledgements

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344 (LNL-JRNL-2012728).